PROJECT-BASED LEARNING (PBL) REPORT · IMPLEMENTATION EDITION

ACADEMIC YEAR 2026–27 · HEALTHCARE INFORMATION SYSTEMS

Online Blood Bank and Donor Management System

A relational database application with role-based dashboards, live stock, request approval and donation records.

STUDENT NAME	[Your Name]
ROLL NUMBER	[Roll No.]
CLASS / DIVISION	[Class & Division]
DEPARTMENT	[Department]
INSTITUTION	[Institution]
FACULTY GUIDE	[Guide Name]
PROJECT DOMAIN	Healthcare Information Systems
TECHNOLOGY	Python · Flask · SQLite · HTML/CSS/JS
SUBMISSION DATE	[DD Month 2026]

Submitted in partial fulfilment of the Project-Based Learning (PBL) component.

All names, hospitals, stock counts and requests in this report are synthetic demonstration data. This prototype is not a clinical system.

Abstract

Blood is a time-critical clinical resource, yet many small blood banks still coordinate donors, stock and hospital requests through disconnected registers, spreadsheets and phone calls. That fragmentation makes it difficult to answer a simple question quickly: which hospital currently has usable units of a given group, and what is the status of an outstanding request?

This project designs and implements an Online Blood Bank and Donor Management System as a working web application. The prototype uses Python 3.10 and Flask 3.1 for the application layer, Jinja templates and a custom CSS design system for the presentation layer, and a normalized SQLite database for storage. The schema contains six related tables — users, hospitals, inventory, requests, donations and appointments — with primary keys, foreign keys, uniqueness rules, check constraints and search indexes. Two roles are supported. Donors register, sign in, check their 90-day eligibility, book a donation slot, search availability by blood group and city, raise requests and review their history. Hospital administrators manage hospitals, stock lines, requests, donations and donor records from a dispatch-board interface.

Requests move through an explicit state machine: Pending → Approved → Completed, with cancellation permitted before fulfilment. The server checks available stock before marking a request complete and deducting units. Security controls include salted password hashing through Werkzeug, session-based authentication, role guards, CSRF protection on every POST, parameterized SQL and server-side validation. The interface was tested at desktop and mobile widths, including a reduced-motion path.

Fourteen functional and security tests and five browser-level checks were executed. All executed checks passed: public pages returned 200, a missing CSRF token returned 400, a donor attempting an administrator page returned 403, an invalid status transition returned 409, completing a request reduced stock from 12 to 11 units, and an SQL-injection-style search returned zero rows. The result is a coherent academic prototype that demonstrates relational design, role-based workflows, validation and interface craft in a socially relevant domain.

Keywords: *blood bank, relational database, Flask, SQLite, CRUD, role-based access, CSRF, normalisation, healthcare information systems.*

SCOPE NOTE

This is an academic prototype. It does not verify medical compatibility, screen donated blood, confirm donor eligibility, or replace hospital procedures. It must not be used for clinical decision-making.

Contents

1. Introduction 4

2. System Analysis 4

3. Requirements 5

4. System Design 6

5. Database Design 8

6. Implementation 10

7. Interface Walkthrough 12

8. Testing and Validation 16

9. Results and Discussion 18

10. Limitations and Future Scope 18

11. Conclusion 19

References 20

Appendix A — Run Instructions and Demo Accounts 21

IMPLEMENTATION NOTE

The original PBL brief specified MySQL and Node.js/Express. The submitted prototype uses Flask and SQLite so that the project runs on a single student laptop without an external database service. The relational schema, keys, joins and constraints are portable to MySQL 8; the change is an engineering decision, not a reduction in database scope.

1. Introduction

1.1 Background

Blood banks sit inside a narrow window between supply and need. A unit has a shelf life, donors have a cooling-off period between donations, and a hospital request may be routine or immediately critical. When donor records, stock sheets and request logs live in separate places, staff spend time reconciling information instead of acting on it. A small, well-structured database application can reduce that friction by giving every desk the same current view.

The project applies relational database principles to this scenario. It is deliberately scoped as a prototype: the goal is to demonstrate a correct schema, enforced relationships, role-based workflows, search and a usable interface, not to reproduce a regulated blood-bank information system.

1.2 Problem Statement

Manual or disconnected records create four recurring problems: duplicate donor entries, stale or inconsistent stock counts, requests whose status is known only to the person who took the call, and searches that depend on memory or a phone tree. The system addresses these by centralising donor, hospital, inventory, request and donation data in one relational store, with one workflow for each record type.

1.3 Objectives

- Design a normalized relational schema with six related entities and explicit integrity constraints.
- Implement donor registration, authentication and a personal dashboard with eligibility and history.
- Give hospital administrators a live board for stock, pending requests, donations and donor records.
- Enforce a request lifecycle in which status changes and stock deductions are validated on the server.
- Provide blood-group and city search that uses parameterised SQL joins rather than string concatenation.
- Protect state-changing operations with session authentication, role guards and CSRF tokens.
- Deliver a responsive, accessible interface that remains usable on a 390px-wide phone.

1.4 Scope

In scope: donor registration and login; donor eligibility and donation history; appointment booking; public and authenticated availability search; donor-raised requests; administrator CRUD for hospitals, inventory, requests and donations; donor directory; role-based dashboards; JSON status updates; and a responsive design system.

Out of scope: clinical compatibility testing, laboratory screening, donor identity verification, payment, SMS/email delivery, live multi-branch synchronisation, regulatory audit trails and production deployment hardening. These exclusions are treated again in Section 10.

2. System Analysis

2.1 Limitations of the Existing Process

- **Fragmented records.** Donor details, stock registers and request books are maintained by different people and rarely share a key.
- **Duplicated donors.** The same person can be registered more than once, which inflates counts and splits donation history.
- **Stale inventory.** Stock is updated after the fact, so a search result may not reflect what is physically on the shelf.

- **Opaque requests.** A requester has no reliable way to see whether a request is pending, approved or fulfilled.
- **Weak reporting.** Aggregating units by group, hospital or city requires manual counting.

2.2 Proposed System

The proposed system is a single web application with two authenticated experiences and one public search surface. A donor owns a profile, donation history, appointments and requests. An administrator owns the hospital directory, stock sheet, request queue, donation log and donor directory. The public can search availability without signing in, but must authenticate to book, request or manage records.

Every screen is generated from the same relational data. The board does not maintain a second copy of stock; it reads the inventory table. A request does not carry a hospital name; it stores a hospital_id and joins to the hospital record. This keeps the interface and the database consistent by construction.

2.3 Users and Roles

Role	Primary goals	Permitted actions
Donor	Find blood, donate, track personal history	Register, sign in, search, book/cancel a slot, raise a request, view own donations and request status
Hospital administrator	Keep stock accurate and clear requests	Manage hospitals and inventory, approve/decline/complete requests, log donations, view donor directory
Public visitor	Check availability before travelling	Search stock by blood group and city; register or sign in

Table 2.1 — User roles and capabilities.

3. Requirements

3.1 Functional Requirements

ID	Requirement	Implementation evidence
FR-01	A visitor can register as a donor with name, email, password, phone, city and blood group.	Registration form; unique email; Werkzeug password hash; automatic sign-in.
FR-02	A registered user can sign in and is routed to the correct dashboard.	Session login; admin → /admin, donor → /dashboard.
FR-03	A donor can see the last donation, next eligible date, donation count and units given.	90-day cooldown calculation in donor_stats().
FR-04	A donor can book or cancel a donation appointment.	Appointment table; future-date validation; ownership check on cancel.
FR-05	Any visitor can search stock by blood group and city.	Parameterized join across inventory and hospitals; results show hospital, city, phone and units.
FR-06	A donor can raise a request for a hospital, group, unit count and urgency.	Validation of group, 1–20 units and urgency; request stored as Pending.
FR-07	An administrator can add hospitals and maintain stock lines.	Hospital form creates eight inventory rows; stock form enforces 0–999 units.
FR-08	An administrator can approve, decline, complete or cancel requests.	Server-enforced state machine; stock check before completion; JSON API.

ID	Requirement	Implementation evidence
FR–09	An administrator can log a donation and update stock.	Donation insert; stock increment for the donor's group at the selected hospital.
FR–10	An administrator can review the donor directory and donation history.	Joined queries with last donation and total donation count.

Table 3.1 — Functional requirements traced to implemented behaviour.

3.2 Non-Functional Requirements

ID	Quality	Target and evidence
NFR–01	Security	Salted password hashing; HttpOnly, SameSite=Lax session cookie; CSRF on all POSTs; role guards; parameterized SQL.
NFR–02	Data integrity	Primary and foreign keys, CHECK constraints, UNIQUE(hospital_id, blood_group), NOT NULL fields.
NFR–03	Usability	Clear navigation, explicit empty states, inline validation, visible request status and destructive-action confirmation copy.
NFR–04	Responsiveness	Desktop 1440px and mobile 390px layouts; no horizontal page overflow at either width.
NFR–05	Accessibility	WCAG AA contrast for text; 40px mobile buttons; keyboard focus rings; reduced-motion path.
NFR–06	Portability	Single Python process with a file-backed database; no external services required for the demo.
NFR–07	Maintainability	One schema file, one stylesheet, one script; reusable Jinja macros and icon set.

Table 3.2 — Non-functional requirements.

3.3 Assumptions and Constraints

- All records are synthetic and clearly labelled “Demo data” in the interface.
- The prototype is single-node and intended for one concurrent demonstration session.
- No medical, laboratory or regulatory validation is performed or implied.
- Passwords must be at least eight characters; the demo accounts use documented credentials.
- The local development server is used without HTTPS; production deployment would require TLS and hardened configuration.

4. System Design

4.1 Architecture

The application follows a three-layer architecture. The presentation layer is server-rendered HTML with a small amount of JavaScript for clocks, counters, toasts, scroll reveals and asynchronous request-status updates. The application layer is a Flask process containing authentication, CSRF protection, role guards, validation and SQL queries. The data layer is SQLite, accessed through Python's sqlite3 module with a row factory and per-request connection.

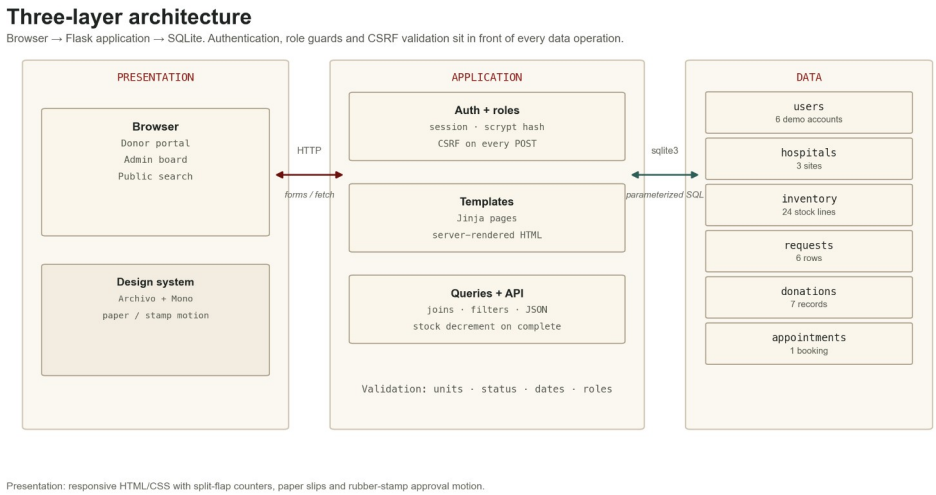


Figure 4.1 — Three-layer architecture. Browser requests pass through authentication, role checks and CSRF validation before parameterized SQL reaches SQLite.

4.2 Module Decomposition

Module	Responsibility	Key routes
Public	Landing page and availability search	/ , /search
Authentication	Registration, login, logout, session and CSRF	/login, /register, /logout
Donor portal	Eligibility, history, appointments, requests	/dashboard, /book, /appointment/<id>/cancel, /request
Admin board	Stock overview, pending request rail, recent donations	/admin
Admin records	Hospitals, inventory, requests, donations, donors	/admin/hospitals, /admin/inventory, /admin/requests, /admin/donations, /admin/donors
Status API	Validated status transitions and stock deduction	/api/requests/<id>/status

Table 4.1 — Application modules.

4.3 Request Lifecycle

A request begins as Pending and appears on the administrator's paper-slip rail. Approval changes the state to Approved but does not move stock: a request is still a request. Completion is the only transition that deducts units, and it is refused if the selected hospital does not hold enough units for that blood group. Cancellation is allowed from Pending or Approved; Completed and Cancelled are terminal.

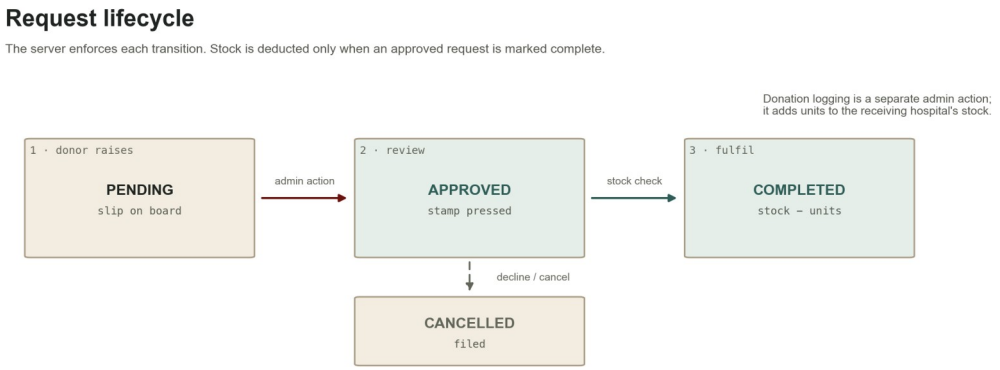


Figure 4.2 — Enforced request lifecycle. Completion performs the stock check and deduction; cancellation is terminal.

4.4 Data Flow

1. A user opens a page or submits a form in the browser.
2. Flask resolves the route and applies session, role and CSRF checks.
3. The view validates input and runs parameterized SQL through the shared query helpers.
4. SQLite returns rows or confirms a write; foreign keys and CHECK constraints reject invalid data.
5. Jinja renders the updated page, or the JSON API returns a status payload to the browser.
6. JavaScript updates a counter, stamp or toast without a full page reload where appropriate.

5. Database Design

5.1 Entity–Relationship Model

The schema contains six entities. Users own requests and donations and may book appointments. Hospitals hold inventory lines, receive requests, receive donations and host appointments. Inventory is unique per hospital and blood group, which prevents duplicate stock rows and makes an upsert-style update safe. Requests and donations reference both a user and a hospital, preserving the history needed to explain a stock movement.

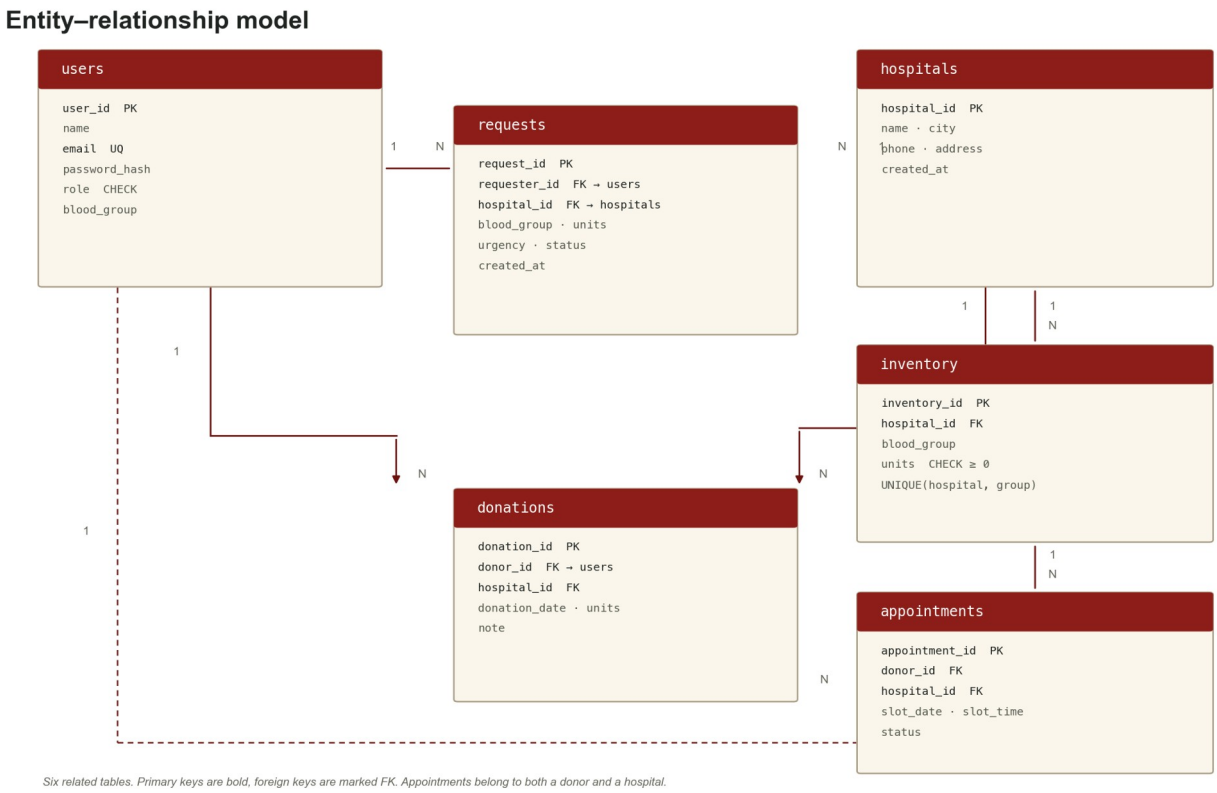


Figure 5.1 — Entity–relationship model with primary keys, foreign keys and one-to-many relationships.

5.2 Tables and Relationships

Table	Purpose	Key fields and relationships
users	One row per donor or administrator	user_id PK; email UNIQUE; role CHECK (donor/admin); blood_group
hospitals	Hospital and blood-centre directory	hospital_id PK; name; city; phone; address
inventory	Stock per hospital and group	inventory_id PK; hospital_id FK → hospitals; UNIQUE(hospital_id, blood_group); units CHECK ≥ 0
requests	Blood requests raised by donors or staff	request_id PK; requester_id FK → users; hospital_id FK → hospitals; status CHECK
donations	Recorded donation events	donation_id PK; donor_id FK → users; hospital_id FK → hospitals; units; date
appointments	Booked donation slots	appointment_id PK; donor_id FK; hospital_id FK; slot_date; slot_time; status

Table 5.1 — Entity overview and key relationships.

5.3 Constraints and Indexes

Constraint	Purpose
PRIMARY KEY	Uniquely identifies every record and gives foreign keys a stable target.
FOREIGN KEY	Prevents inventory, requests, donations or appointments from referencing a missing user or hospital.
UNIQUE(email)	Prevents duplicate donor accounts.

Constraint	Purpose
`UNIQUE(hospital_id, blood_group)`	Guarantees one stock line per group per hospital.
`CHECK(units ≥ 0) / CHECK(units > 0)`	Rejects negative stock and non-positive request or donation quantities.
`CHECK(role) and CHECK(status)`	Restricts roles and workflow states to known values.
`ON DELETE CASCADE (inventory)`	Removes stock lines only when their parent hospital is deleted; request history blocks deletion first.
`idx_inventory_group, idx_requests_status, idx_donations_donor`	Speed up the most frequent filters and joins.

Table 5.2 — Integrity rules and indexes.

5.4 Normalisation

The schema is in Third Normal Form. Each field is atomic (1NF). Stock, requests and donations use whole-row primary keys, so no non-key attribute depends on only part of a candidate key (2NF). Descriptive data is stored once: a request stores hospital_id rather than hospital name or city, and the display joins to hospitals. A donor's blood group lives in users, not repeated in every donation row. Removing any of these indirect dependencies would lose meaning, so no transitive dependency remains (3NF).

Inventory is the one intentional exception to a pure event log: it stores the current count, while donations and requests provide the history that explains changes. This is a deliberate read-performance choice for a dashboard, and the trade-off is documented rather than hidden.

5.5 History and Deletion

Healthcare records should not disappear casually. Hospitals with requests cannot be deleted; the administrator receives an explanatory message. Inventory rows cascade only when their parent hospital is legitimately removed. Requests and donations are retained in their terminal states, so the board can still explain why stock changed. A production system would add soft deletion, audit tables and role-specific retention rules.

6. Implementation

6.1 Technology Stack

Layer	Technology	Why it was chosen
Language	Python 3.10	Readable server code and a mature standard library.
Web framework	Flask 3.1.3	Small surface area; explicit routes; easy to demonstrate in a PBL setting.
Templates	Jinja2	Server-rendered HTML with reusable macros and autoescaping.
Database	SQLite 3	Zero-configuration file database; supports foreign keys, CHECK constraints and joins.
Front end	HTML5, CSS3, vanilla JavaScript	No build step; full control over the dispatch-board visual language.
Security	Werkzeug password hashing, signed sessions, CSRF tokens	Standard, well-understood primitives without custom cryptography.
Testing	Flask test client, Playwright (Chrome)	Route-level assertions and real-browser layout/contrast checks.

Table 6.1 — Implementation stack.

DEVIATION FROM THE ORIGINAL BRIEF

The brief named MySQL 8 and Node.js/Express. The implemented prototype uses SQLite and Flask so it can run from a single folder on a student machine. The data model, keys, constraints and join queries map directly to MySQL; Section 10 lists the migration path.

6.2 Project Structure

```
blood-bank-app/
├── app.py                 # schema, seed, routes, auth, API
├── templates/             # Jinja pages and reusable macros
├── static/css/style.css   # design system and motion
├── static/js/app.js       # clock, counters, stamps, toasts
├── static/fonts/          # local Archivo and JetBrains Mono
└── instance/bloodbank.db  # SQLite database (created on first run)
```

6.3 Authentication, Roles and CSRF

Registration validates the email, password length, blood group and required fields, hashes the password with Werkzeug and stores the new user with the donor role. Login verifies the hash and writes user_id and role into the signed session. Two decorators protect the application: login_required redirects anonymous visitors to the sign-in page, while admin_required returns 403 for a signed-in donor who tries to open an administrator route.

Every POST is checked by a before_request hook. The token is generated per session, embedded in forms and in a meta tag for fetch calls, and compared using secrets.compare_digest. The API accepts the token through the X-CSRF-Token header; a missing or stale token returns HTTP 400 with a JSON error.

```
@app.before_request
def csrf_protect():
    if request.method == "POST":
        sent = request.form.get("csrf_token") or request.headers.get("X-CSRF-Token")
        if not sent or not secrets.compare_digest(sent, session.get("csrf", "")):
            if request.path.startswith("/api/"):
                return jsonify({"error": "Invalid CSRF token."}), 400
            abort(400)
```

6.4 Request State Machine and Stock Check

Status changes are not accepted as free text. The API defines the allowed next states for each current state and returns HTTP 409 for anything else. Completion performs a stock read for the request's hospital and group; if the row is missing or holds fewer units than requested, the transition is refused. Only then does the inventory row decrement and the request become Completed. The browser receives the new status and the updated total for that blood group, which drives the counter animation.

```
allowed = {
    "Pending":   {"Approved", "Cancelled"},
    "Approved":  {"Completed", "Cancelled"},
    "Completed": set(),
    "Cancelled": set(),
}
# ...
if new_status == "Completed":
    inv = get_inventory(row["hospital_id"], row["blood_group"])
    if not inv or inv["units"] < row["units"]:
        return jsonify({"error": "Not enough units in stock."}), 409
    deduct_units(inv["inventory_id"], row["units"])
```

6.5 Search and Query Design

Search is built from optional predicates rather than one fixed query. A blood group uses exact equality; a city uses a parameterized LIKE. The query joins inventory to hospitals so a result can show the site name, city and phone without duplicating those fields in the inventory table. Only rows with positive stock are returned, ordered by available units and hospital name.

```
SELECT i.blood_group, i.units, h.name, h.city, h.phone
FROM inventory i
JOIN hospitals h ON h.hospital_id = i.hospital_id
WHERE i.blood_group = ?
  AND h.city LIKE ?
  AND i.units > 0
ORDER BY i.units DESC, h.name;
```

Values are always bound as parameters. A search for a quote-based injection payload is treated as a literal city string and returns no rows, as recorded in Table 8.2.

6.6 Interface and Motion

The interface is built around a dispatch-board metaphor rather than a generic admin template: cream paper, an oxidized-red rail, ink typography, amber signals and teal approvals. Archivo carries the interface; JetBrains Mono carries references, dates and stock counts. Stock is shown as paper strips with split-flap counters; pending requests are paper slips; approval presses a rubber stamp and then lifts the slip away.

Motion is purposeful and short: a 260ms counter tick, a 620ms stamp, a 320ms toast and a 60ms stagger on scroll reveals. The reduced-motion media query collapses animation and transition durations and reveals all sections immediately. Nothing relies on animation to communicate state; the status text and stock value are always present.

6.7 Validation and Error Handling

- Required fields, email format, password length and blood-group membership are checked server-side.
- Units are bounded to 1–20 for requests and 0–999 for stock; appointments reject past dates.
- Ownership checks prevent a donor from cancelling another donor's appointment.
- 403 and 404 pages use the same visual world and offer a route back to the board.
- Flash messages confirm successful writes and explain refused operations in plain language.

7. Interface Walkthrough

The following screenshots were captured from the running application with the seeded demo database. They are evidence of the rendered interface, not mock-ups.

7.1 Public Landing and Availability Search

The landing page states the problem and offers two immediate actions: search availability or register as a donor. The search form accepts a blood group and a city; the result row gives the hospital, city, phone number and units, so a visitor can act without signing in. The footer exposes group-level totals and flags low groups.

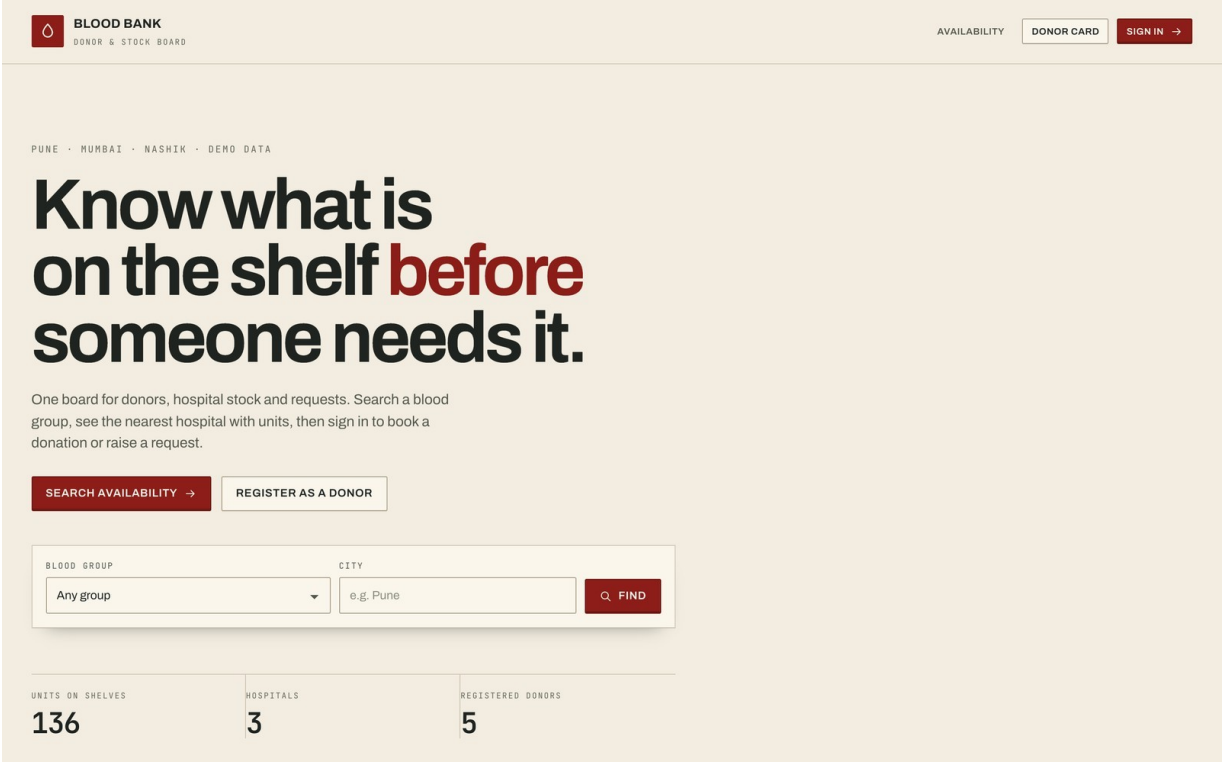


Figure 7.1 — Public landing page at 1440×900. The board metaphor is established before any data is shown.

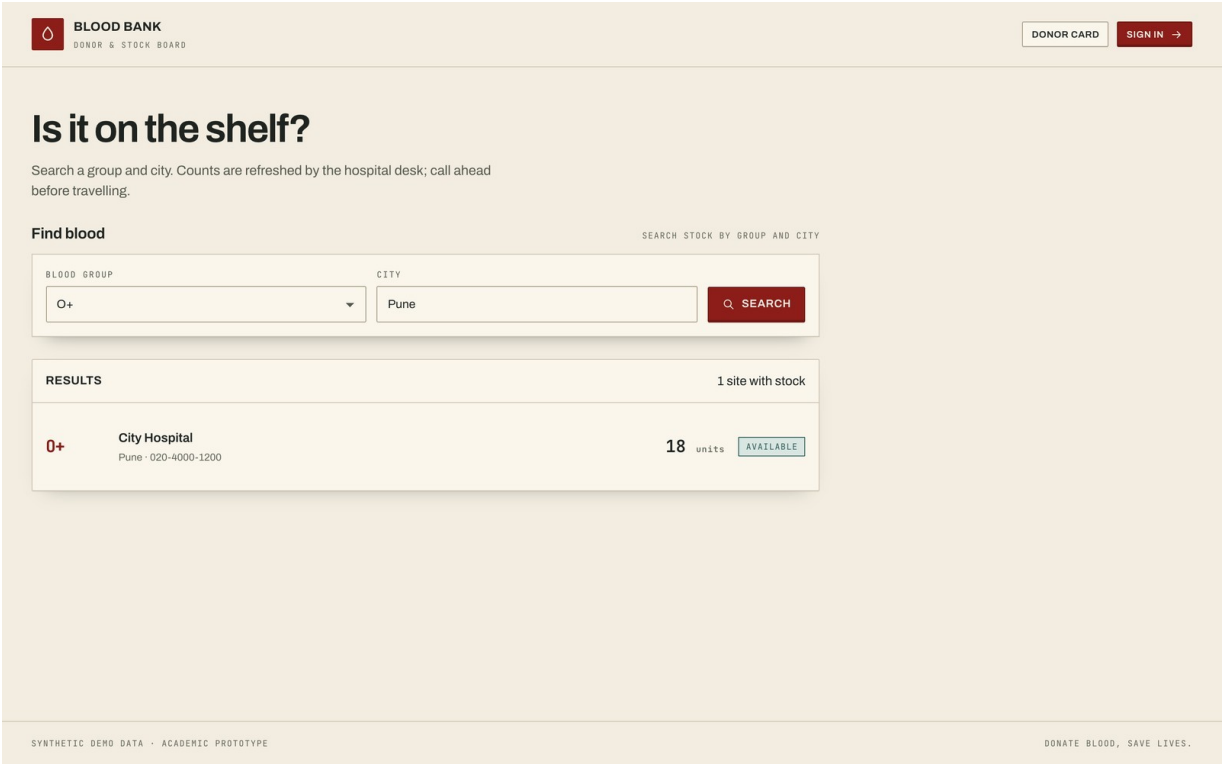


Figure 7.2 — Public availability search for O+ in Pune, returning City Hospital with its contact number.

7.2 Administrator Dispatch Board

The board is the administrator's first viewport. Group stock is arranged as split-flap strips, with critical groups marked in red. Pending requests sit in a paper-slip rail; each slip carries the reference, group, units, hospital, urgency and the two decisions. The right column shows recent donations and hospital stock totals. The top statistics summarise units, critical groups, pending requests and registered donors.

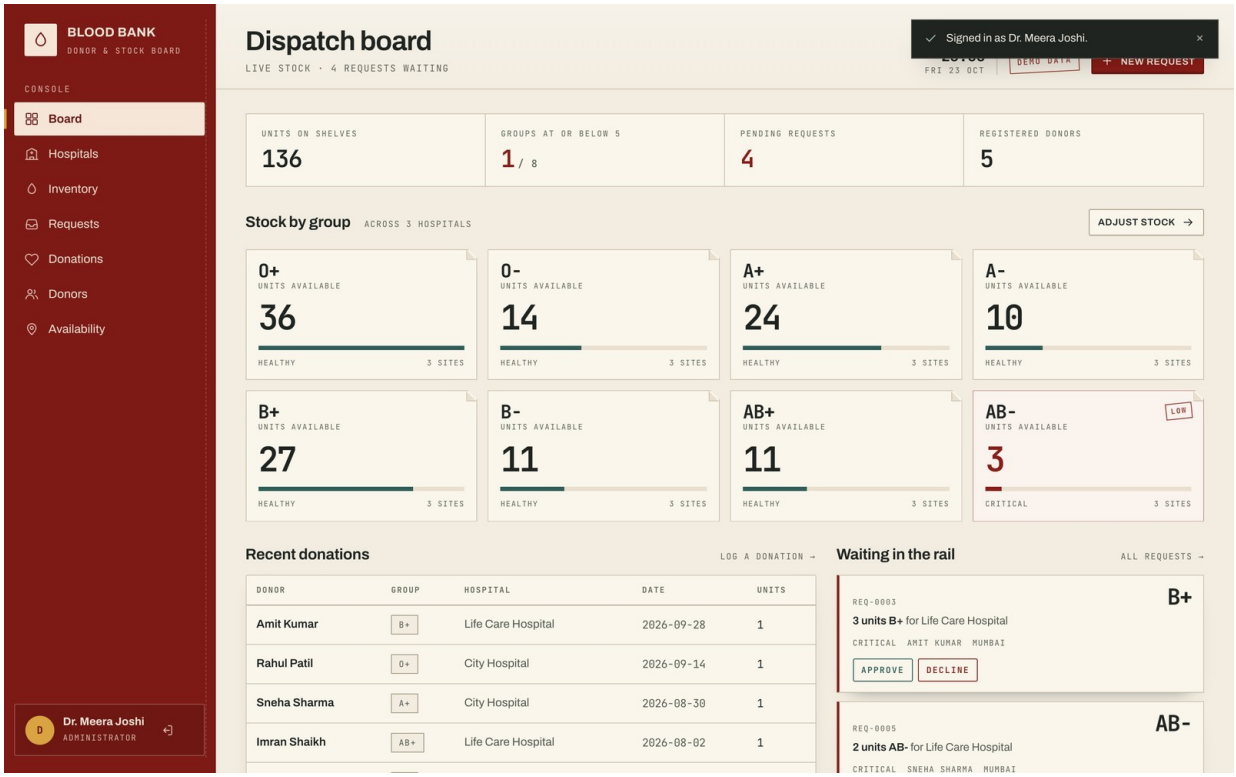


Figure 7.3 — Administrator dispatch board at 1440×900, showing stock strips, pending request slips and recent donations.

7.3 Request Rail and Status Actions

The request page exposes the full queue with status filters. Pending requests offer Approve and Decline; approved requests offer Complete and Cancel. Completing a request is the only action that changes stock, and the confirmation is shown as a toast while the stock counter animates. Terminal rows are retained with a Filed label.

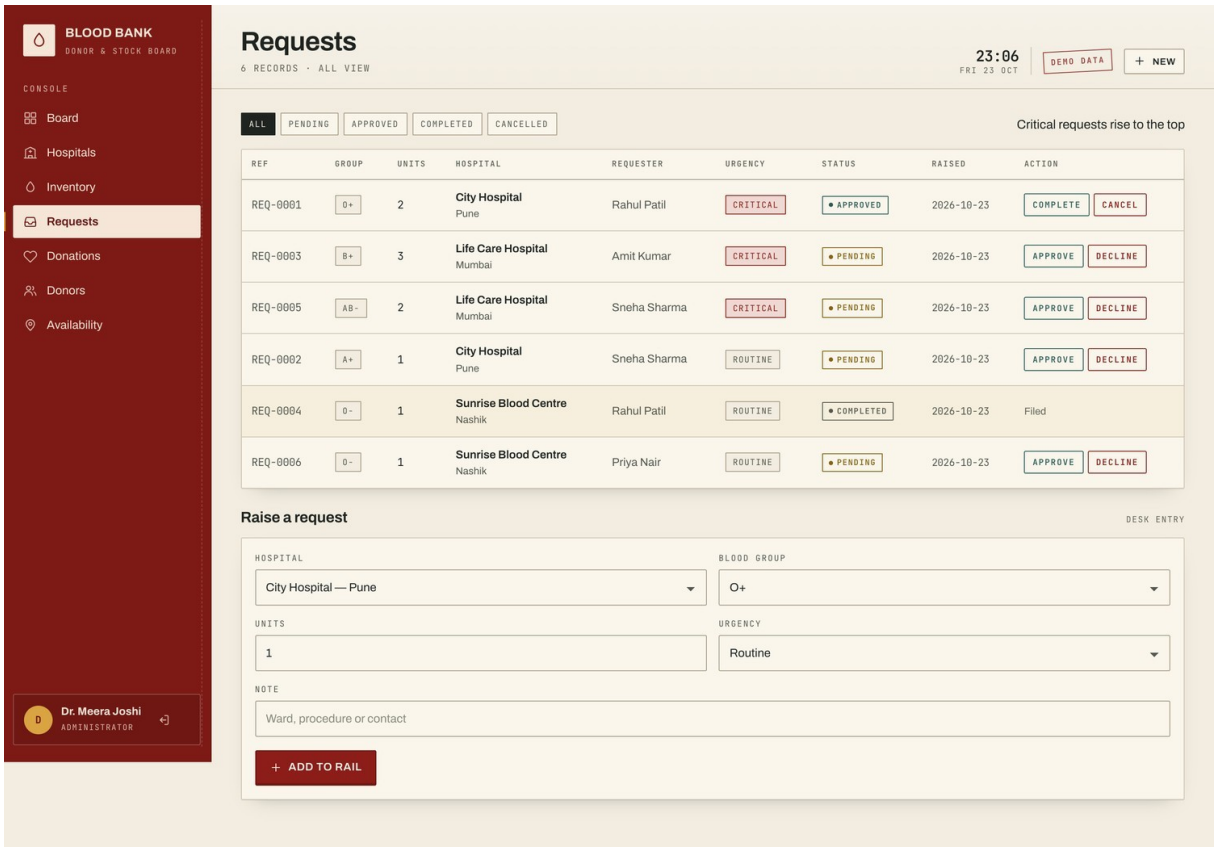


Figure 7.4 — Request queue with urgency, status, hospital and permitted actions.

7.4 Donor Dashboard

The donor dashboard answers the questions a donor actually asks: when did I last donate, when can I donate again, what have I given, and what is happening to my request? Eligibility is calculated from the most recent donation plus the 90-day cooldown. The same page carries appointment booking, a request form and personal request history.

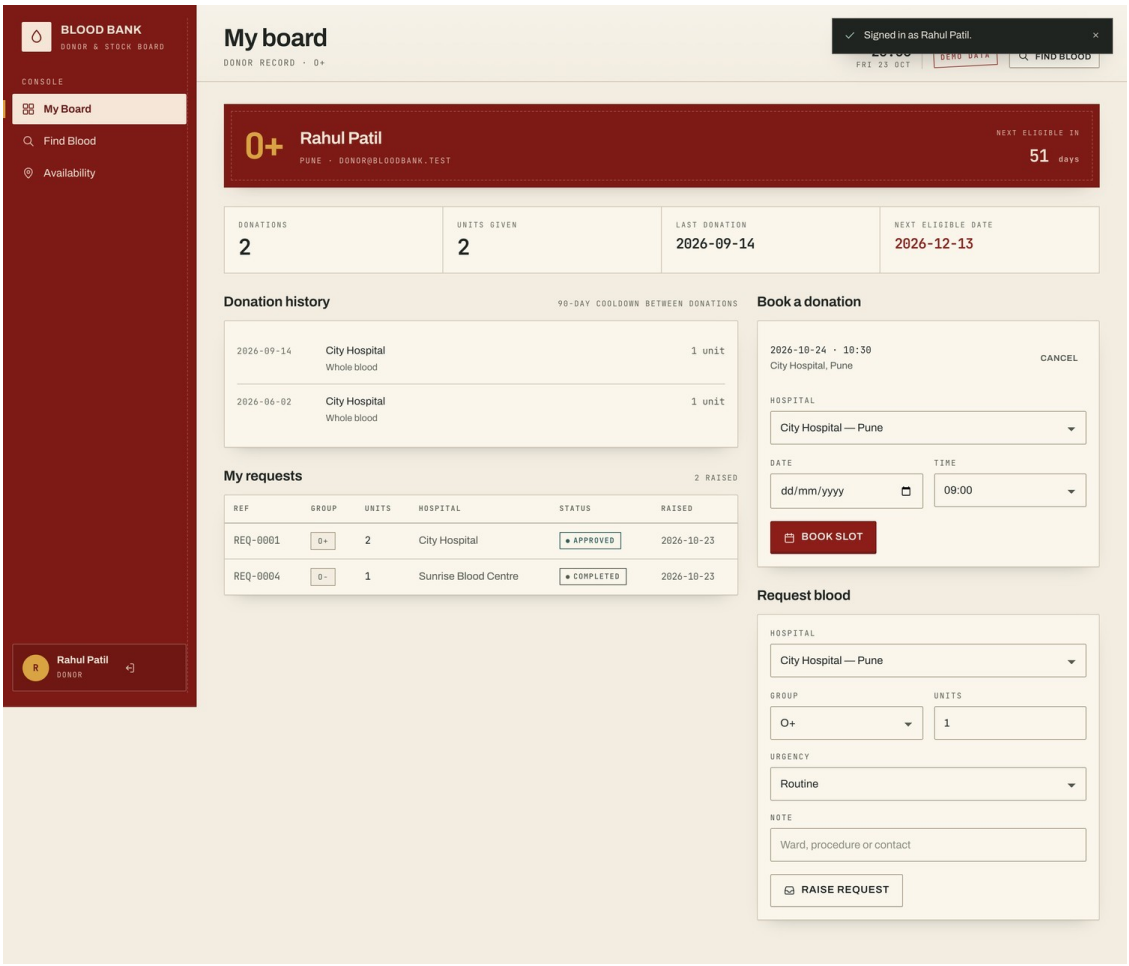


Figure 7.5 — Donor dashboard with eligibility strip, donation history, appointment booking and request history.

7.5 Responsive Behaviour

At 390px the rail becomes a compact header with a horizontally scrollable tab strip, the board collapses to one column, stock strips become two-up, and action buttons grow to a 40px minimum height. The mobile layout has no horizontal page overflow; wide tables scroll inside their own container. The active tab is automatically centred on load.

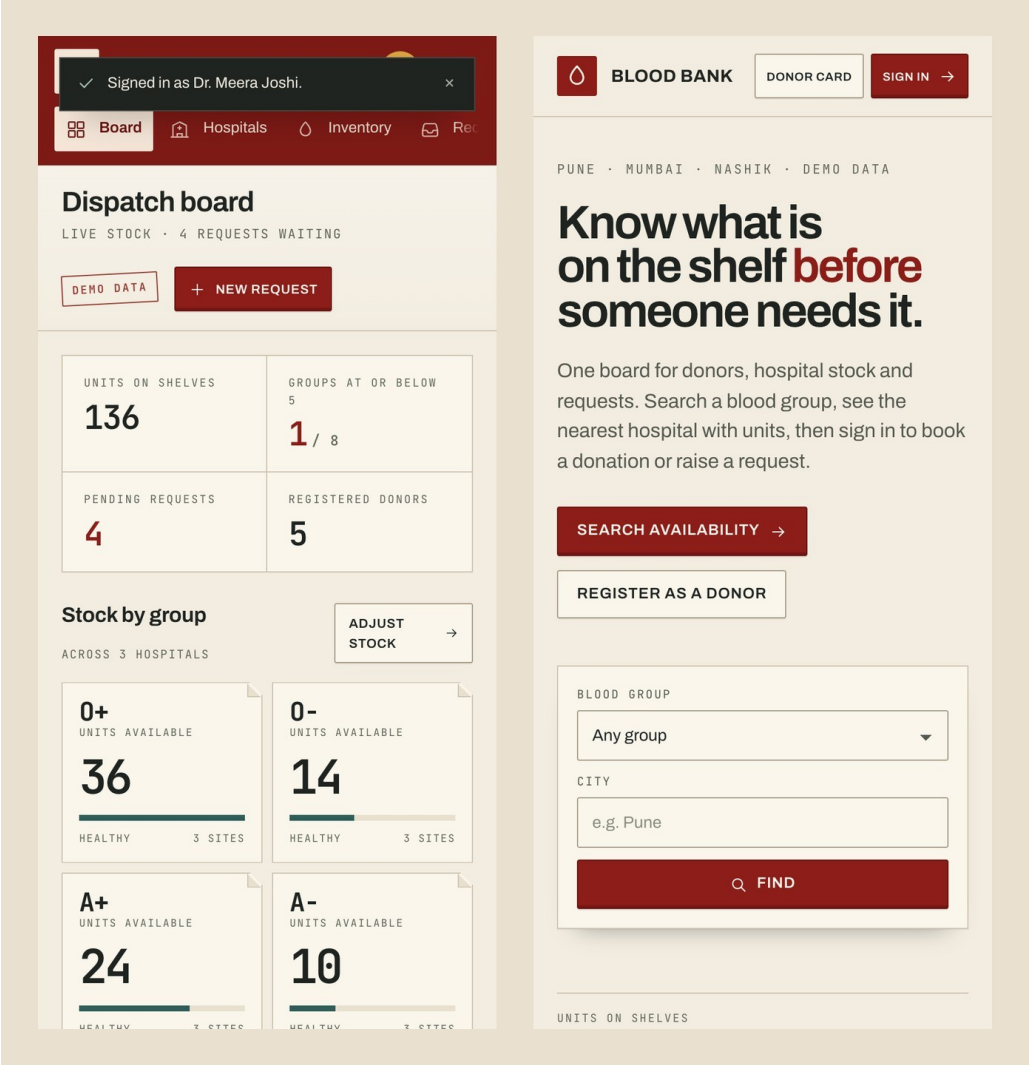


Figure 7.6 — Mobile layout at 390×844: administrator board (left) and public landing page (right).

8. Testing and Validation

8.1 Test Environment

Item	Value
Application	Flask 3.1.3, Python 3.10.18, SQLite 3
Database state	Seeded demo database: 1 administrator, 5 donors, 3 hospitals, 24 stock lines, 136 units, 6 requests, 7 donations, 1 appointment
Route tests	Flask test client with a fresh seeded database
Browser tests	Google Chrome via Playwright at 1440×900 and 390×844
Checks	HTTP status, database row counts, stock before/after, transition errors, horizontal overflow, contrast, reduced motion

Table 8.1 — Test environment and starting data.

8.2 Functional and Security Tests

ID	Test	Expected	Actual	Result
T01	Open public landing page	200 and rendered board	200	Pass
T02	Search O+ in Pune	Matching hospital rows only	200; City Hospital returned	Pass
T03	POST login without CSRF token	Rejected	400	Pass
T04	Admin login	Redirect to /admin	302 → /admin	Pass
T05	Donor login	Redirect to /dashboard	302 → /dashboard	Pass
T06	Donor opens /admin	Forbidden	403	Pass
T07	Donor calls status API	Forbidden	403	Pass
T08	Approve a pending request	Status Approved; stock unchanged	200; Approved	Pass
T09	Complete an approved request	Status Completed; stock deducted	200; 12 → 11 units	Pass
T10	Repeat a completed transition	Rejected by state machine	409	Pass
T11	Raise a valid donor request	New Pending row	302; count 6 → 7	Pass
T12	Book a future appointment	New Booked row	302; count 1 → 2	Pass
T13	SQL-injection-style city search	Treated as data; no leak	200; 0 result rows	Pass
T14	Cancel another donor's appointment	Not found / refused	404	Pass (ownership check)

Table 8.2 — Executed functional and security tests. The database was reset after the test run.

8.3 Browser and Accessibility Checks

Check	Method	Result
Horizontal overflow at 1440px	document.scrollWidth vs clientWidth	No overflow
Horizontal overflow at 390px	document.scrollWidth vs clientWidth	No page overflow; tables scroll internally
Text contrast	Computed foreground/background luminance	All sampled text meets WCAG AA
Reduced motion	prefers-reduced-motion: reduce	Counters show final values; reveals instant
Loaded assets	Chrome console and network	No application errors; local fonts loaded

Table 8.3 — Browser-level checks.

8.4 Discussion of Results

The executed tests cover the paths most likely to fail in a role-based CRUD application: authentication, authorisation, CSRF, state transitions, stock consistency and search safety. The most important result is not that a page rendered, but that the server refused invalid actions. A donor could not reach an administrator page, a completed request could not be approved again, and stock changed only when a valid completion was processed.

The browser checks address a different risk: a desktop-only interface that looks correct in a screenshot but breaks on a phone or excludes a keyboard user. The mobile navigation, table wrappers and minimum

button sizes were adjusted after finding a 20px overflow and several sub-38px controls; the final build reports no page-level overflow and passes sampled contrast checks. The test set is deliberately small and transparent, not an exhaustive penetration test or clinical validation.

9. Results and Discussion

The prototype meets the seven objectives stated in Section 1.3. The schema is normalized, the relationships are enforced by the database, and the application exposes a coherent workflow for each role. The result is best read as a chain of evidence: schema → route → validation → rendered state → recorded test.

Objective	Evidence
Relational schema	Six tables with primary keys, foreign keys, CHECK and UNIQUE constraints; ER model in Figure 5.1.
Donor portal	Registration and login; eligibility, history, appointment booking and personal requests on one dashboard.
Administrator board	Stock strips, pending slips, recent donations, hospital totals and request queue.
Server-enforced lifecycle	Allowed transitions table; 409 on invalid transition; stock check before deduction.
Parameterized search	Join query with bound parameters; injection-style input returned zero rows.
Security controls	Password hashing; role guards return 403; missing CSRF token returns 400.
Responsive, accessible UI	390px layout without page overflow; WCAG AA contrast; reduced-motion support.

Table 9.1 — Objectives mapped to implementation evidence.

Two results are worth highlighting. First, the stock completion test produced a measurable state change (12 → 11 units) rather than a cosmetic success message, which demonstrates that the interface and the database are connected through the validated API. Second, the authorisation tests returned 403 rather than a redirect to a generic page, which shows that the role check is enforced at the route boundary.

The design also contributes to the result. A conventional dashboard would have displayed the same data, but the dispatch-board language gives the administrator a mental model that matches the work: stock is a strip, a request is a slip, approval is a stamp, and completion is a deduction. That metaphor is not decoration; it makes the state of the system legible at a glance.

10. Limitations and Future Scope

10.1 Limitations

- SQLite is a single-node database. It is adequate for a demonstration but not for concurrent multi-branch operation.
- The prototype does not verify clinical compatibility, screen blood, confirm donor identity or enforce medical eligibility.
- There is no email/SMS notification, audit trail, rate limiting or production-grade backup strategy.
- Search is not paginated; a large inventory would need limits, total counts and cursor-based navigation.
- The local server runs over HTTP; deployment would require TLS, secure cookies, a production WSGI server and secret management.

- Automated tests cover core workflows but are not a substitute for a full accessibility audit or penetration test.

10.2 Future Scope

- Migrate the schema to MySQL 8 or PostgreSQL and add versioned migrations for multi-user deployment.
- Add audit logging and soft deletion for donor, inventory and request changes.
- Introduce notification workflows for request status, appointment reminders and low-stock alerts.
- Add pagination, saved searches and CSV export for administrators.
- Integrate a map or geo-search for nearby hospitals, with consent and privacy controls.
- Expand automated tests into a CI pipeline, including accessibility checks and load testing.
- Add configurable inventory thresholds and expiry tracking for blood units.

11. Conclusion

This project turns a relational database exercise into a working healthcare workflow. The six-table schema captures donors, hospitals, stock, requests, donations and appointments with explicit keys and constraints. The Flask application gives each role a focused surface: donors can find blood and manage their own contribution history; administrators can see the shelf, clear the request rail and record donations. The request state machine ensures that stock moves only after a valid, checked completion.

The implementation also demonstrates that database quality is visible in the interface. When relationships are enforced, a hospital name cannot drift between screens; when stock is read from one table, the counter and the completion result agree; when CSRF and role guards are present, an invalid action is refused rather than silently ignored. These are the behaviours the tests exercised.

The prototype is not a clinical product. It is an academic demonstration of schema design, server-side validation, role-based access and interface craft in a domain where those qualities matter. With the migration, audit and notification work described in Section 10, the same structure could serve as the foundation for a larger blood-bank information system.

References

[1] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th ed. Pearson, 2017.

[2] Pallets Projects, Flask Documentation (3.1.x), 2026. [Online]. Available: https://flask.palletsprojects.com/

[3] SQLite Consortium, SQLite Documentation, 2026. [Online]. Available: https://www.sqlite.org/docs.html

[4] Mozilla Foundation, MDN Web Docs: HTML, CSS and JavaScript, 2026. [Online]. Available: https://developer.mozilla.org/

[5] OWASP Foundation, Cross-Site Request Forgery Prevention Cheat Sheet, 2025. [Online]. Available: https://cheatsheetseries.owasp.org/

[6] W3C, Web Content Accessibility Guidelines (WCAG) 2.2, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

[7] World Health Organization, Blood safety and availability, 2024. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability

[8] Werkzeug Documentation, Password Hashing, 2026. [Online]. Available: https://werkzeug.palletsprojects.com/

Appendix A — Run Instructions and Demo Accounts

A.1 Local Setup

```
cd blood-bank-app
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:5000
```

The database is created and seeded automatically on first run in instance/bloodbank.db. To reset the demonstration, stop the server, delete that file and start the server again.

A.2 Demo Accounts

Role	Email	Password
Administrator	admin@bloodbank.test	Admin@123
Donor	donor@bloodbank.test	Donor@123
Donor (secondary)	sneha@bloodbank.test	Donor@123

Table A.1 — Demo credentials. All accounts and records are synthetic.

A.3 Data Reset

Deleting instance/bloodbank.db and restarting the application recreates the schema and reseeds the demonstration. No production data is used at any point.

ACADEMIC INTEGRITY

The implementation, screenshots and test results in this report describe the submitted prototype. Where the implementation differs from the original design brief — notably Flask and SQLite instead of Express and MySQL — the deviation is stated explicitly in Section 6.1 rather than hidden.